



TÀI LIỆU ÔN TẬP PHÒNG VẤN CHUYÊN SÂU: TECH STACK TOÀN DIỆN

Đối tượng: Kỹ sư Tốt nghiệp loại Giỏi Kỹ thuật Phần mềm (Software Engineering)

Định vị ứng viên: Fresher / Junior Backend Developer (NestJS/NodeJS), Full-stack Engineer (Next.js/React), AI Application & Cloudflare Serverless Engineer

Nguyên tắc trả lời phỏng vấn: Không học vẹt định nghĩa — Hiểu sâu bản chất cơ chế hoạt động, phân tích ưu nhược điểm (Trade-offs) và liên hệ thực tế dự án đã làm.

MỤC LỤC

- [Module 1: TypeScript & JavaScript Core \(Nền Tảng, Cốt Lõi\).](#)
- [Module 2: NestJS & Node.js Backend Architecture](#)
- [Module 3: Database & ORM \(MySQL, PostgreSQL, Prisma, Cloudflare D1\).](#)
- [Module 4: Redis & Bull Queue \(Caching & Asynchronous Processing\).](#)
- [Module 5: Next.js 15 & Modern Frontend Architecture](#)
- [Module 6: Cloudflare Serverless & Docker DevOps](#)
- [Module 7: AI & LLM Application Engineering](#)
- [Module 8: Git Flow, Code Review & Verification Discipline](#)

MODULE 1: TYPESCRIPT & JAVASCRIPT CORE

1. Event Loop trong Node.js hoạt động như thế nào? Phân biệt Microtask và Macrotask.

- Bản chất:** Node.js là Single-Threaded Event Loop được vận hành bởi thư viện **libuv**.
- Các pha của Event Loop:**
 - Timers:** Thực thi callbacks của `setTimeout()` và `setInterval()`.
 - Pending Callbacks:** Thực thi I/O callbacks bị hoãn lại từ vòng lặp trước.
 - Idle, Prepare:** Sử dụng nội bộ bởi libuv.
 - Poll:** Đọc dữ liệu I/O mới từ hệ điều hành (file system, socket mạng).
 - Check:** Thực thi callbacks của `setImmediate()`.
 - Close Callbacks:** Xử lý các kết nối đóng như `socket.on('close')`.
- Microtask vs Macrotask:**
 - Microtask Queue:** Gồm `process.nextTick()` (ưu tiên cao nhất trong Node.js) và `Promise.then()` / `catch` / `finally`, `queueMicrotask()`.
 - Quy tắc quan trọng:** Microtask Queue luôn được rút cạn (drain) ngay sau khi tác vụ hiện tại kết thúc và **TRƯỚC KHI** Event Loop chuyển sang pha tiếp theo.

- **Hệ quả:** Nếu đệ quy gọi `process.nextTick()`, bạn sẽ làm "đói" (starve) toàn bộ I/O của server.

2. Sự khác biệt giữa `interface` và `type` trong TypeScript? Khi nào nên dùng cái nào?

- **Điểm giống nhau:** Dùng để định nghĩa hình dạng (shape) của dữ liệu và kiểm tra kiểu tĩnh.
- **Điểm khác nhau:**

Tiêu chí	<code>interface</code>	<code>type</code>
Declaration Merging	Hỗ trợ gộp tự động khi khai báo trùng tên (thường dùng để extend thư viện bên thứ ba)	Báo lỗi duplicate identifier
Union / Primitives	Không tạo được union type trực tiếp (<code>type Status = 'PENDING' 'DONE'</code>)	Hỗ trợ Union, Intersection, Primitives, Tuples
Kế thừa	Dùng từ khóa <code>extends</code>	Dùng toán tử giao <code>&</code>
Hiệu năng Compiler	Compiler tối ưu bộ nhớ cache interface nhanh hơn một chút	Chậm hơn nhẹ khi xử lý complex conditional types

- **Quy tắc thực tế:**
 - Dùng `interface` cho Object models, DTOs, Services, Controller contracts trong NestJS.
 - Dùng `type` cho Union types, Utility types, Tuples, hoặc Function signatures phức tạp.

3. Phân biệt `any`, `unknown` và `never` trong TypeScript.

- `any`: Tắt hoàn toàn trình kiểm tra kiểu (Type checker). Cho phép gán bất cứ thứ gì và gọi bất cứ hàm nào mà không bị bắt lỗi compile-time -> **Tránh dùng tối đa trong production.**
- `unknown`: Kiểu an toàn của `any`. Có thể nhận bất kỳ giá trị nào, nhưng **bắt buộc phải Type Narrowing** (dùng `typeof`, `instanceof` hoặc user-defined type guard) trước khi thao tác hoặc gọi method.
- `never`: Biểu diễn giá trị **không bao giờ xảy ra**. Dùng cho hàm luôn ném Exception (`throw new Error()`), hàm có vòng lặp vô tận, hoặc dùng trong Exhaustive Checking của câu lệnh `switch-case`.

4. Các Utility Types quan trọng thường dùng trong dự án là gì?

- `Partial<T>`: Chuyển toàn bộ thuộc tính của `T` thành optional (thường dùng trong Update DTO).
 - `Required<T>`: Bắt buộc toàn bộ thuộc tính.
 - `Pick<T, K>`: Chỉ chọn một tập thuộc tính `K` từ `T`.
 - `Omit<T, K>`: Loại bỏ các thuộc tính `K` khỏi `T` (ví dụ: `Omit<User, 'passwordHash'>`).
 - `Record<K, V>`: Tạo Object Map với key kiểu `K` và value kiểu `V`.
 - `ReturnType<T>`: Trích xuất kiểu trả về của một function.
-

MODULE 2: NESTJS & NODE.JS BACKEND ARCHITECTURE

1. Vòng đời Request (Request Lifecycle) trong NestJS diễn ra theo thứ tự nào?

Thứ tự chuẩn (Ghi nhớ để trả lời ngay lập tức):

1. **Incoming Request**
2. **Global / Module / Controller Middleware** (Logging, Cors, Body parser)
3. **Guards** (Global -> Controller -> Route) (Xác thực JWT, kiểm tra Role RBAC)
4. **Interceptors (Pre-controller)** (Đo lường thời gian, biến đổi input)
5. **Pipes** (Global -> Controller -> Route -> Param) (Validation DTO với `ValidationPipe`, `ParseIntPipe`)
6. **Controller Action** (Route Handler)
7. **Service** (Xử lý Business Logic)
8. **Interceptors (Post-request)** (Biến đổi response payload, format chuẩn `{ data, statusCode }`)
9. **Exception Filters** (Nếu có lỗi throw ra: bắt và format chuẩn mã lỗi HTTP JSON)
10. **Outgoing Response**

2. Dependency Injection (DI) và Inversion of Control (IoC) trong NestJS là gì?

- **IoC (Inversion of Control)**: Thay vì class tự mình `new` các dependency (ví dụ: `new PrismaService()`), việc khởi tạo và quản lý vòng đời của các dependency được đảo ngược lại cho framework (IoC Container của NestJS) quản lý.
- **DI (Dependency Injection)**: Là cơ chế cụ thể để thực thi IoC. Service được đánh dấu bằng `@Injectable()`, và được inject vào constructor của Controller hoặc Service khác thông qua cơ chế Type reflection của TypeScript.
- **Lợi ích**:
 - Giảm độ phụ thuộc cứng (Loose Coupling).
 - Cực kỳ dễ dàng viết Unit Test vì có thể dễ dàng mock các service phụ thuộc (`Test.createTestingModule()`).

3. Phân biệt các Scopes của Provider trong NestJS (`DEFAULT`, `REQUEST`, `TRANSIENT`).

- `Scope.DEFAULT` (Khuyến nghị): **Singleton pattern**. Một instance duy nhất được tạo ra lúc ứng dụng khởi động và dùng chung cho toàn bộ requests. Hiệu năng cao nhất, tiết kiệm RAM.
- `Scope.REQUEST`: Mỗi HTTP Request đến sẽ tạo ra một instance mới và hủy đi khi request kết thúc. **Cảnh báo**: Tốn kém tài nguyên và làm chậm tốc độ phản hồi nếu dùng bừa bãi (chỉ dùng khi cần lưu trữ thông tin riêng của request như request-scoped logging).
- `Scope.TRANSIENT`: Mỗi class inject provider này sẽ nhận một instance hoàn toàn riêng biệt.

4. DTO và Validation trong NestJS được triển khai như thế nào?

- Sử dụng **Data Transfer Object (DTO)** định nghĩa dạng `class` kết hợp thư viện `class-validator` và `class-transformer`.
- Kích hoạt toàn cục `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })`:
 - `whitelist: true`: Tự động loại bỏ các trường thừa không nằm trong DTO (chống tấn công Over-posting).
 - `transform: true`: Tự động ép kiểu dữ liệu từ chuỗi sang kiểu dữ liệu khai báo trong DTO.

MODULE 3: DATABASE & ORM (MYSQL, POSTGRESQL, PRISMA, D1)

1. Sự khác biệt giữa Clustered Index và Non-Clustered Index?

- **Clustered Index (Chỉ mục cụm):**
 - Xác định **thứ tự vật lý** lưu trữ dữ liệu thực tế trên đĩa cứng (Leaf nodes của B-Tree chứa toàn bộ dòng dữ liệu thật).
 - Mỗi bảng chỉ có duy nhất **1 Clustered Index** (thường mặc định là Khóa chính - Primary Key).
- **Non-Clustered Index (Chỉ mục không cụm / Secondary Index):**
 - Được lưu trữ ở một cấu trúc riêng biệt. Leaf nodes chỉ chứa giá trị của cột được index và **con trỏ (pointer/row ID/Primary key)** trỏ về dòng dữ liệu trong bảng chính.
 - Một bảng có thể có nhiều Non-Clustered Index.
 - Khi query không nằm trong chỉ mục bao phủ (Covering Index), database phải thực hiện thêm bước tra cứu lại bảng gốc (**Bookmark Lookup / Key Lookup**).

2. Vấn đề N+1 Query trong ORM là gì? Cách khắc phục trong Prisma?

- **Bản chất:** Xảy ra khi ta muốn lấy một danh sách \$N\$ đối tượng cha và mỗi đối tượng cha cần lấy danh sách con liên quan:
 - Query 1: `SELECT * FROM Author` (Lấy \$N\$ tác giả).
 - Query 2...N+1: Trong vòng lặp, chạy tiếp \$N\$ câu query `SELECT * FROM Book WHERE authorId = ?` để lấy sách của từng tác giả.
 - Tổng cộng: $\$1 + N\$$ queries gửi về database -> Làm nghẽn I/O nghiêm trọng.
- **Cách khắc phục trong Prisma:**
 - Sử dụng mệnh đề `include` hoặc `select`:

```
const authors = await prisma.author.findMany({
  include: { books: true },
});
```

- Prisma sẽ tự động tối ưu thành **2 câu truy vấn duy nhất**:
 1. `SELECT * FROM Author`
 2. `SELECT * FROM Book WHERE authorId IN (id1, id2, id3, ...)` Sau đó Prisma tự ghép nối (in-memory join) trong code Node.js, loại bỏ triệt để N+1 query.

3. Phân biệt 4 cấp độ cô lập dữ liệu (Transaction Isolation Levels) trong SQL.

1. **Read Uncommitted:** Thấp nhất. Có thể đọc dữ liệu chưa commit của transaction khác -> Bị lỗi **Dirty Read**.
2. **Read Committed:** Chỉ đọc dữ liệu đã commit -> Tránh được Dirty Read, nhưng có thể bị **Non-Repeatable Read** (đọc lại cùng 1 dòng thì giá trị đã bị transaction khác sửa).
3. **Repeatable Read** (Mặc định của MySQL InnoDB): Đảm bảo trong suốt transaction đọc lại một dòng thì dữ liệu không đổi -> Tránh Non-repeatable read, nhưng có thể gặp **Phantom Read** (xuất hiện dòng mới chèn vào thỏa mãn điều kiện WHERE).

4. **Serializable**: Cao nhất và an toàn nhất. Khóa toàn bộ các thao tác tuần tự -> Hiệu năng thấp nhất, dễ gây Deadlock nếu lưu lượng cao.
-

MODULE 4: REDIS & BULL QUEUE (CACHING & ASYNCHRONOUS)

1. Phân biệt Cache Aside, Write-Through và Write-Back?

- **Cache-Aside (Lazy Loading)** (Phổ biến nhất):
 - Ứng dụng đọc: Đọc Cache trước -> Nếu trúng (Cache Hit) thì trả về; Nếu hụt (Cache Miss) thì query Database, ghi ngược lại vào Cache rồi trả về.
 - Ứng dụng ghi: Ghi trực tiếp vào Database, sau đó **xóa khóa Cache tương ứng (Cache Invalidation)** để lần đọc sau tự nạp dữ liệu mới.
- **Write-Through**: Dữ liệu luôn được ghi vào Cache trước, Cache tự đồng bộ ghi vào DB trước khi trả về thành công.
- **Write-Back (Write-Behind)**: Ghi vào Cache tức thì, định kỳ batch job ghi dồn vào DB phía sau (nhanh nhất nhưng nguy cơ mất dữ liệu nếu Cache sập).

2. Các thảm họa Caching thường gặp và cách phòng chống?

- **Cache Penetration (Xuyên thủng Cache)**: Kẻ tấn công query liên tục các key không hề tồn tại trong DB -> Request xuyên thẳng xuống DB làm sập DB.
 - *Giải pháp*: Lưu trữ key rỗng với TTL ngắn (`set key null ex 60s`) hoặc dùng **Bloom Filter**.
- **Cache Breakdown (Gây Cache / Hotkey hết hạn)**: Một key cực kỳ hot (hàng triệu người đọc) đột ngột hết hạn TTL đúng lúc -> Hàng nghìn request đồng loạt tràn xuống DB query cùng 1 lúc.
 - *Giải pháp*: Dùng **Mutual Exclusion Mutex Lock** (chỉ cho 1 request lấy khóa xuống DB nạp lại cache) hoặc làm mới cache logic bất đồng bộ trước khi hết hạn.
- **Cache Avalanche (Tuyết lở Cache)**: Hàng loạt các key trong cache cùng hết hạn TTL tại một thời điểm hoặc Redis bị restart -> Toàn bộ request dồn xuống DB.
 - *Giải pháp*: Thêm **thời gian ngẫu nhiên (Jitter / Random TTL)**: `TTL = 3600 + Math.random() * 300`.

3. Bull Queue (BullMQ) quản lý vòng đời Job như thế nào?

- **Các trạng thái Job**:
 - `Waiting` -> `Active` (Worker lấy xử lý) -> `Completed` / `Failed`.
 - Nếu Failed và có cấu hình retry: Chuyển sang `Delayed` chờ hết thời gian Exponential Backoff trước khi quay lại `Waiting`.
 - **Idempotency trong Queue**:
 - Luôn gán `jobId` duy nhất (ví dụ: `generate-exam-${examSessionId}`). Nếu client bấm 2 lần, Redis kiểm tra `jobId` đã tồn tại sẽ từ chối tạo job trùng lặp.
-

MODULE 5: NEXT.JS 15 & MODERN FRONTEND

1. Phân biệt React Server Components (RSC) và Client Components trong Next.js 15 App Router?

- **React Server Components (RSC)** (Mặc định trong App Router):
 - Render hoàn toàn trên Server, không gửi mã nguồn JavaScript của component đó về trình duyệt.
 - Có thể trực tiếp truy cập Database, filesystem, API secrets an toàn.
 - Giảm kích thước bundle JS gửi về client (Zero bundle size impact).
 - **Hạn chế:** Không dùng được React hooks (`useState` , `useEffect`) và browser event listeners (`onClick`).
- **Client Components** (Khai báo `'use client'` ở đầu file):
 - Vẫn được pre-render thành HTML trên server, sau đó trình duyệt thực hiện quá trình **Hydration** để gắn các event listeners.
 - Dùng cho giao diện tương tác người dùng, forms, state management, animations.

2. Lỗi Hydration Mismatch trong Next.js là gì và nguyên nhân phổ biến?

- **Bản chất:** Xảy ra khi cây HTML được tạo ra từ Server không khớp chính xác 100% với cây Virtual DOM được React render lần đầu ở Client.
 - **Nguyên nhân phổ biến:**
 - Dùng các API chỉ có trên Client như `window` , `localStorage` , `navigator` trực tiếp trong giai đoạn render ban đầu.
 - Dùng `new Date().toLocaleTimeString()` hoặc `Math.random()` dẫn đến kết quả ở Server và Client khác nhau.
 - Cấu trúc thẻ HTML không hợp lệ (ví dụ lồng thẻ `<p>` bên trong một thẻ `<p>` khác, hoặc lồng `<div>` bên trong `<p>`).
 - **Khắc phục:** Đưa logic phụ thuộc client vào trong `useEffect()` hoặc dùng dynamic import `{ ssr: false }` .
-

MODULE 6: CLOUDFLARE SERVERLESS & DOCKER DEVOPS

1. So sánh Cloudflare Workers (V8 Isolates) với Node.js Container truyền thống.

Tiêu chí	Cloudflare Workers (V8 Isolates)	Docker Container (Node.js)
Cơ chế chạy	Hàng nghìn isolates chạy chung trong 1 tiến trình V8 process được cách ly bộ nhớ	Mỗi container là một hệ điều hành thu nhỏ với Node runtime riêng
Cold Start	0ms - 5ms (Khởi tạo tức thì)	500ms - 3000ms (Khởi động hệ điều hành + nạp node_modules)
Tài nguyên RAM	Cực kỳ nhẹ (~ vài MB cho mỗi isolate)	Tốn kém (~ 50MB - 200MB cho mỗi container)
Hạ tầng phân tán	Tự động phân tán trên 275+ Edge PoPs toàn cầu	Phải tự dựng cụm Cluster (Kubernetes / ECS)
Hạn chế	Không dùng được các thư viện phụ thuộc C++ addons hoặc filesystem Node.js thuần	Hỗ trợ 100% tính năng Node.js và C++ bindings

2. Multi-Stage Build trong Dockerfile là gì? Tại sao phải dùng?

- Vấn đề:** Trong quá trình build NestJS/Next.js, ta cần các công cụ nặng như `typescript`, `@types/*`, compiler tools. Nếu giữ toàn bộ trong image cuối, image sẽ phình to từ 1GB đến 2GB, tốn thời gian kéo image (pull) và tiềm ẩn lỗ hổng bảo mật.
- Giải pháp (Multi-stage Build):**
 - Stage 1 (Builder):** Cài đặt đầy đủ `devDependencies`, chạy `npm run build` tạo ra thư mục `dist`.
 - Stage 2 (Runner):** Sử dụng base image siêu nhẹ (ví dụ `node:20-alpine`), chỉ copy thư mục `dist` và cài đặt `dependencies` phục vụ runtime (`npm ci --only=production`).
- Kết quả:** Giảm kích thước Docker image từ **1.5GB xuống còn dưới 120MB**, tăng tốc CI/CD và giảm diện tích bề mặt tấn công bảo mật.

MODULE 7: AI & LLM APPLICATION ENGINEERING

1. Kỹ thuật RAG (Retrieval-Augmented Generation) hoạt động như thế nào?

- Indexing Phase:**
 - Văn bản tài liệu được chia nhỏ thành các đoạn (**Text Chunks**).
 - Đưa qua mô hình Embedding để chuyển đổi thành các vector số học nhiều chiều (ví dụ 128-dim hoặc 1536-dim).
 - Lưu trữ các vector vào cơ sở dữ liệu vector (Vector DB).
- Retrieval Phase:**
 - Người dùng đặt câu hỏi -> Vector hóa câu hỏi thành vector truy vấn.

- Tính toán độ tương đồng ngữ nghĩa bằng thuật toán **Cosine Similarity**:
$$\frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$
- Lấy ra top K đoạn văn bản có độ tương đồng cao nhất.

3. Generation Phase:

- Gắn các đoạn văn bản vừa tìm được vào Context của Prompt gửi cho LLM.
- LLM tổng hợp và sinh câu trả lời dựa trên căn cứ thực tế (Grounding), loại bỏ ảo giác.

2. Sự khác biệt giữa Function Calling (Tools) và Prompt Chay?

- **Prompt Chay**: Ép LLM sinh văn bản tự do, dễ bị ảo giác, không có khả năng tương tác với thế giới thực bên ngoài.
- **Function Calling (Tools)**:
 - Lập trình viên định nghĩa danh sách hàm với JSON Schema (tên hàm, mô tả, các tham số bắt buộc).
 - Mô hình LLM không tự chạy hàm. Nó chỉ đóng vai trò phân tích xem câu nói của người dùng tương ứng với hàm nào, và **trích xuất tham số chuẩn xác dưới dạng JSON**.
 - Ứng dụng Backend nhận JSON, trực tiếp thực thi hàm (gọi DB, gọi API thanh toán) và trả kết quả về cho LLM tóm tắt lại cho người dùng.

MODULE 8: GIT FLOW & VERIFICATION DISCIPLINE

1. Phân biệt `git merge` và `git rebase`. Khi nào nên dùng cái nào?

- `git merge`: Tạo ra một **Merge Commit** mới kết hợp lịch sử của hai nhánh.
 - *Ưu điểm*: Lưu giữ nguyên vẹn 100% lịch sử thực tế của các nhánh con.
 - *Nhược điểm*: Khi có nhiều nhánh nhỏ, cây git log sẽ bị rối rắm (nhiều nhánh đan xen).
- `git rebase`: Nhỏ toàn bộ các commit của nhánh hiện tại và "cắm" lại vào ngọn của nhánh đích.
 - *Ưu điểm*: Tạo ra lịch sử commit hoàn toàn thẳng hàng (linear history), cực kỳ sạch sẽ và dễ đọc.
 - *Nguyên tắc vàng*: **Không bao giờ rebase trên các nhánh công khai dùng chung (như `main` hoặc `develop`),** chỉ rebase trên nhánh feature cá nhân trước khi mở Pull Request.

2. Quy chuẩn PR Proof of Work mà bạn áp dụng có gì khác biệt với việc tạo PR thông thường?

- **PR thông thường (Thiếu minh chứng)**: Thường chỉ tạo PR với tiêu đề ngắn, vài dòng commit sơ sài, không chạy thử trên môi trường thật, đẩy rủi ro kiểm thử cho đồng đội hoặc tester.
- **PR Proof of Work (Quy chuẩn kỷ luật)**:
 - Phân nhánh `feature/*` rõ ràng, commit atomic theo Conventional Commits.
 - Bắt buộc chạy `tsc -b` pass 0 lỗi biên dịch.
 - **Bắt buộc đính kèm ảnh chụp màn hình UI hoặc video screen recording luồng nghiệp vụ thực tế vào Markdown description của PR trên GitHub.**
 - Báo cáo rõ ràng: Những gì ĐÃ kiểm chứng vs Những gì CHƯA kiểm thử.
 - Giúp Tech Lead nghiệm thu tức thì trong 30 giây mà không cần pull code về chạy lại từ đầu.